

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Khoa Toán - Cơ - Tin học

BÁO CÁO THỰC TẬP DOANH NGHIỆP

Năm học 2025 – 2026

AI-ECDSA Digital Signature System

Đường cong Elliptic và ứng dụng trong chữ ký số
tích hợp AI phát hiện IP nghi vấn

Sinh viên thực hiện: Phúc Nguyễn Dương - 22000113

Hoàng Xuân Đức - 22000129

Nguyễn Công Trường - 22000113

Lớp: K67A2 Toán - Tin

Đơn vị thực tập:

Giảng viên hướng dẫn: Nguyễn Duy Niên

Chức vụ: Senior Security Engineer

Hà Nội, tháng 5 năm 2026

Lời cảm ơn

Chúng em xin gửi lời cảm ơn chân thành đến:

- **Thầy Nguyễn Duy Niên** — Senior Security Engineer, người đã tận tình định hướng về lý thuyết mật mã học đường cong Elliptic, hỗ trợ hạ tầng môi trường kiểm thử và review toàn bộ mã nguồn trong suốt quá trình thực tập. Những góp ý của thầy về kiến trúc module và bảo mật CSPRNG là nền tảng quan trọng để chúng em hoàn thiện hệ thống.
- **Trường Đại học Quốc gia Hà Nội** — đã tạo môi trường học thuật, trang thiết bị nghiên cứu và tài nguyên tài liệu phong phú để chúng em có thể thực hiện dự án này.

Tóm tắt

Báo cáo trình bày quá trình nghiên cứu, thiết kế và triển khai hệ thống **AI-ECDSA Digital Signature System** — một ứng dụng mật mã học thuần Python kết hợp trí tuệ nhân tạo. Hệ thống bao gồm ba thành phần chính: (1) module lõi mật mã tự triển khai từ đầu (*from scratch*) các thuật toán ECDSA, ECGDSA và EC ElGamal trên 11 đường cong SEC2 chuẩn; (2) module AI bảo mật *IP Guardian* sử dụng Isolation Forest kết hợp cơ chế hard rules để phát hiện IP nghi vấn theo thời gian thực; và (3) giao diện người dùng đồ họa ba tab xây dựng bằng Tkinter.

Về kết quả định lượng: trên đường cong secp256r1, thời gian ký số trung bình đạt **~42 ms**, xác thực đạt **~85 ms**; trên secp112r1 (đường cong nhẹ nhất), ký chỉ mất **~5 ms**. Module AI phát hiện tấn công thành công trong **100%** các kịch bản kiểm thử (rate-limit, fail-rate, burst) với thời gian phản hồi dưới **1 ms**. Bộ unit test đạt **100%** pass trên toàn bộ 14 test case.

Từ khóa: Elliptic Curve Cryptography (ECC), ECDSA, ECGDSA, EC ElGamal, Isolation Forest, chữ ký số, bảo mật IP.

Mục lục

Lời cảm ơn	i
Tóm tắt	ii
1 Giới thiệu	1
1.1 Bối cảnh và động lực	1
1.2 Đóng góp chính của sinh viên	1
1.3 Mục tiêu dự án	1
1.4 Phạm vi và giới hạn	2
1.5 Cấu trúc báo cáo	2
2 Cơ sở lý thuyết	3
2.1 Lý do lựa chọn ECC thay vì RSA	3
2.2 Mật mã đường cong Elliptic (ECC)	3
2.2.1 Định nghĩa đường cong Elliptic	3
2.2.2 Phép toán trên đường cong	4
2.2.3 Bài toán ECDLP và nền tảng bảo mật	4
2.3 Thuật toán ECDSA	4
2.3.1 Tạo cặp khóa	4
2.3.2 Ký số	4
2.3.3 Xác thực	5
2.4 Thuật toán ECGDSA	5
2.5 Mã hóa EC ElGamal	5
2.6 Isolation Forest cho phát hiện bất thường IP	5
2.6.1 Lý do chọn Isolation Forest	5
2.6.2 Nguyên lý hoạt động	6
2.6.3 Lựa chọn ngưỡng 0.55	6
3 Kiến trúc hệ thống	7
3.1 Tổng quan kiến trúc phân lớp	7
3.2 Cấu trúc thư mục	7
3.3 Module <code>src/core</code> — Lõi mật mã	8
3.3.1 Luồng xử lý ECDSA end-to-end	8
3.3.2 Luồng sử dụng EC ElGamal trong giao diện	8
3.4 Module <code>src/ai</code> — AI IP Guardian	9
3.4.1 Kiến trúc 3 lớp bảo vệ	9
3.4.2 7 đặc trưng đầu vào	10

3.5	Đường cong SEC2 được hỗ trợ	10
4	Triển khai chi tiết	11
4.1	Module point.py — Phép toán nguyên thủy	11
4.1.1	Lớp Point và xử lý trường hợp biên	11
4.1.2	Thuật toán Double-and-Add	11
4.2	Module ecdsa.py — Chữ ký số ECDSA & ECGDSA	12
4.2.1	Triển khai ECDSA với CSPRNG	12
4.2.2	So sánh ECDSA và ECGDSA	14
4.3	Module ec_elgamal.py — Mã hóa EC ElGamal	14
4.4	Module utils.py — Tiện ích mật mã	14
5	Kiểm thử và đánh giá	16
5.1	Môi trường kiểm thử	16
5.2	Kiểm thử module mật mã (test_ecdsa.py)	16
5.2.1	Kết quả unit test	16
5.2.2	Benchmark hiệu năng theo đường cong	17
5.3	Kiểm thử AI IP Guardian (test_ip_guardian.py)	18
5.3.1	Kết quả unit test	18
5.3.2	Mô phỏng kịch bản tấn công brute-force	18
5.3.3	Đánh giá mô hình Isolation Forest	18
5.4	Giao diện người dùng và UX	19
5.4.1	Tab 1 — Chữ ký số (ECDSA / ECGDSA)	19
5.4.2	Tab 3 — AI IP Guardian	19
6	Kết luận	20
6.1	Kết quả đạt được so với mục tiêu ban đầu	20
6.2	Hạn chế của hệ thống	20
6.3	Hướng phát triển tiếp theo	21
7	Đánh giá quá trình thực tập	22
7.1	Kiến thức và kỹ năng tiếp thu được	22
7.2	Những điều học được tại môi trường thực tập	22
7.3	Tự đánh giá	22

Danh sách hình vẽ

3.1	Sơ đồ thành phần (component diagram) của hệ thống AI-ECDSA Digital Signature System	7
3.2	Luồng thực thi thuật toán ECDSA ký số	8
3.3	Luồng xử lý 3 lớp bảo vệ của AI IP Guardian	9
5.1	Hiệu năng ECDSA theo kích thước đường cong (ms)	17

Danh sách bảng

2.1	So sánh ECC và RSA về kích thước khóa và hiệu năng	3
2.2	So sánh các phương pháp phát hiện bất thường	6
3.1	Các đặc trưng đầu vào của mô hình Isolation Forest	10
3.2	11 đường cong SEC2 trong <code>curves_db.py</code>	10
4.1	So sánh ECDSA và ECGDSA	14
5.1	Môi trường kiểm thử thực nghiệm	16
5.2	Kết quả kiểm thử ECDSA / ECGDSA / ElGamal (tham chiếu Liệt kê 4.3) . . .	16
5.3	Hiệu năng thực đo ECDSA (trung bình 5 lần đo, máy tham chiếu Bảng 5.1) . .	17
5.4	Kết quả kiểm thử AI IP Guardian (tham chiếu Hình 3.3)	18
5.5	Diễn biến phát hiện tấn công brute-force	18
5.6	Thành phần tập kiểm thử AI IP Guardian	18
5.7	Chỉ số đánh giá mô hình Isolation Forest trên tập kiểm thử mô phỏng	19
6.1	Đối chiếu mục tiêu và kết quả	20

Listings

4.1	Phép cộng điểm – <code>point_add()</code> (<code>point.py</code>)	11
4.2	Nhân vô hướng $k \cdot P$ — Double-and-Add (<code>point.py</code>)	11
4.3	Tạo khóa và ký số ECDSA (<code>ecdsa.py</code>)	12
4.4	Xác thực chữ ký ECDSA (<code>ecdsa.py</code>)	13
4.5	Mã hóa và giải mã EC ElGamal (<code>ec_elgamal.py</code>)	14
4.6	Kiểm tra nguyên tố Miller-Rabin (<code>utils.py</code>)	14

Chương 1

Giới thiệu

1.1 Bối cảnh và động lực

Trong kỷ nguyên số hóa, bảo mật thông tin trở thành thách thức hàng đầu. Mật mã đường cong Elliptic (ECC — *Elliptic Curve Cryptography*) cung cấp mức bảo mật tương đương RSA nhưng với khóa ngắn hơn đáng kể: khóa 256-bit ECC tương đương bảo mật với khóa RSA 3072-bit [?]. Đồng thời, các cuộc tấn công mạng ngày càng tinh vi đòi hỏi cơ chế phát hiện bất thường thông minh và tự động.

Dự án **AI-ECDSA Digital Signature System** được thực hiện trong khuôn khổ thực tập tại Đại học Quốc gia Hà Nội năm 2026, với mục tiêu vừa củng cố nền tảng toán học về ECC, vừa xây dựng một hệ thống bảo mật hoàn chỉnh có khả năng ứng dụng trong môi trường nghiên cứu và giáo dục.

1.2 Đóng góp chính của sinh viên

Các đóng góp được thực hiện hoàn toàn độc lập bởi sinh viên, trừ phần tham khảo tài liệu chuẩn:

1. **Tự triển khai thư viện ECC thuần Python** (không dùng thư viện mật mã bên ngoài): bao gồm phép cộng điểm, nhân vô hướng Double-and-Add, nghịch đảo modular bằng Extended Euclidean Algorithm, kiểm tra tham số miền.
2. **Cài đặt ba thuật toán mật mã**: ECDSA, ECGDSA và EC ElGamal theo đúng đặc tả chuẩn FIPS 186-4, SEC2, BSI TR-03111.
3. **Thiết kế kiến trúc 3 lớp bảo vệ** cho module AI IP Guardian (Whitelist, Hard Rules, Isolation Forest) và tự xây dựng tập dữ liệu mô phỏng tấn công.
4. **Xây dựng giao diện người dùng** ba tab bằng Tkinter, tích hợp đầy đủ các luồng: tạo khóa, ký, xác thực, mã hóa, giải mã, giám sát IP.
5. **Thiết kế bộ unit test** 14 test case bao phủ các tình huống bình thường, biên và tấn công.

1.3 Mục tiêu dự án

1. Xây dựng thư viện mật mã ECC thuần Python hỗ trợ 11 đường cong SEC2.
2. Triển khai ECDSA, ECGDSA và EC ElGamal đầy đủ, sử dụng CSPRNG (secrets) để đảm bảo an toàn mật mã học.

3. Phát triển module AI phát hiện IP nghi vấn theo thời gian thực.
4. Xây dựng giao diện đồ họa trực quan, dễ sử dụng.
5. Kiểm thử toàn diện và đánh giá hiệu năng theo từng đường cong.

1.4 Phạm vi và giới hạn

Hệ thống tập trung vào:

- Triển khai các phép toán ECC trong trường hữu hạn $\mathbb{F}_p$.
- Sử dụng SHA-512 (512-bit) làm hàm băm nhất quán cho toàn hệ thống.
- Isolation Forest với 7 đặc trưng hành vi IP.

Hệ thống **không** nhằm thay thế các thư viện mật mã chuẩn trong môi trường production (vì không chống side-channel attack, chưa benchmark trên dữ liệu production thực tế), mà phục vụ mục đích giáo dục và nghiên cứu.

1.5 Cấu trúc báo cáo

Chương 2 — Cơ sở lý thuyết

Nền tảng toán học ECC, lý do chọn các thuật toán, so sánh với RSA, nguyên lý Isolation Forest.

Chương 3 — Kiến trúc hệ thống

Cấu trúc module, luồng dữ liệu, sơ đồ thành phần.

Chương 4 — Triển khai chi tiết

Giải thích các đoạn code quan trọng, quyết định kỹ thuật, lưu ý bảo mật.

Chương 5 — Kiểm thử và đánh giá

Unit tests, benchmark hiệu năng, thực nghiệm AI.

Chương 6 — Kết luận

Tổng kết, hạn chế, hướng phát triển.

Chương 7 — Đánh giá quá trình thực tập

Kết quả đạt được so với mục tiêu, bài học kinh nghiệm.

Chương 2

Cơ sở lý thuyết

2.1 Lý do lựa chọn ECC thay vì RSA

Trước khi đi vào chi tiết toán học, cần làm rõ lý do chọn ECC làm nền tảng cho hệ thống. Bảng 2.1 so sánh hai hướng tiếp cận:

Bảng 2.1: So sánh ECC và RSA về kích thước khóa và hiệu năng

Mức bảo mật	Khóa RSA (bit)	Khóa ECC (bit)	Tỉ lệ
80-bit	1024	160	$\approx 6:1$
112-bit	2048	224	$\approx 9:1$
128-bit	3072	256	$\approx 12:1$
192-bit	7680	384	$\approx 20:1$
256-bit	15360	521	$\approx 30:1$

Khóa nhỏ hơn đồng nghĩa: ít bộ nhớ lưu trữ, truyền dữ liệu nhanh hơn, và tính toán nhanh hơn — đặc biệt có lợi trên thiết bị IoT và hệ thống nhúng. Lý do chọn **SHA-512** (thay vì SHA-256) là để đảm bảo kích thước hash đủ lớn so với modulus của các đường cong lớn nhất (secp521r1, 521-bit).

2.2 Mật mã đường cong Elliptic (ECC)

2.2.1 Định nghĩa đường cong Elliptic

Định nghĩa 2.1. Đường cong Elliptic trên trường hữu hạn $\mathbb{F}_p$ (với p nguyên tố) được định nghĩa bởi phương trình Weierstrass rút gọn:

$$E : y^2 \equiv x^3 + ax + b \pmod{p}$$

kèm điểm vô cực $\mathcal{O}$, với điều kiện không suy biến $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

Tập hợp các điểm trên đường cong cùng phép cộng tạo thành nhóm Abel hữu hạn $(E(\mathbb{F}_p), +)$. Phần tử trung hòa là $\mathcal{O}$.

2.2.2 Phép toán trên đường cong

Phép cộng điểm ($P + Q = R$): cho $P = (x_1, y_1)$, $Q = (x_2, y_2)$, hệ số góc λ được tính khác nhau tùy hai điểm có bằng nhau không:

$$\lambda = \begin{cases} \frac{3x_1^2 + a}{2y_1} \bmod p & \text{nếu } P = Q \text{ (nhân đôi điểm)} \\ \frac{y_2 - y_1}{x_2 - x_1} \bmod p & \text{nếu } P \neq Q \end{cases}$$

$$x_3 = \lambda^2 - x_1 - x_2 \bmod p, \quad y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$$

Nhân vô hướng ($k \cdot P$): thực hiện bằng thuật toán Double-and-Add với độ phức tạp $O(\log_2 k)$.

2.2.3 Bài toán ECDLP và nền tảng bảo mật

Toàn bộ bảo mật của ECC dựa trên **bài toán logarithm rời rạc trên đường cong Elliptic** (ECDLP): cho $Q = k \cdot G$ với G là điểm sinh công khai, việc tìm lại k từ G và Q là bài toán tính toán khó — thuật toán tốt nhất hiện tại (Pollard's rho) có độ phức tạp $O(\sqrt{n})$, với n là bậc của nhóm. Đây là lý do tại sao khóa ECC 256-bit cung cấp bảo mật tương đương RSA 3072-bit: không gian tìm kiếm hiệu quả là tương đương nhau.

2.3 Thuật toán ECDSA

ECDSA (*Elliptic Curve Digital Signature Algorithm*) là chuẩn chữ ký số theo FIPS 186-4 [1].
Trực giác: người ký tạo ra một “bằng chứng” dựa trên khóa riêng d và một nonce bí mật k — chữ ký (r, s) — người xác thực có thể kiểm tra bằng khóa công khai $Q = d \cdot G$ mà không cần biết d hay k .

2.3.1 Tạo cặp khóa

1. Chọn ngẫu nhiên $d \in [1, n - 1]$ (khóa riêng), dùng CSPRNG.
2. Tính $Q = d \cdot G$ (khóa công khai).

2.3.2 Ký số

Cho thông điệp m , khóa riêng d :

1. Tính $h = \text{SHA-512}(m) \bmod n$.
2. Chọn ngẫu nhiên $k \in [1, n - 1]$ (nonce bí mật, dùng secrets).
3. $R = k \cdot G = (x_R, y_R)$; $r = x_R \bmod n$. Nếu $r = 0$ thì chọn k khác.
4. $s = k^{-1}(h + d \cdot r) \bmod n$. Nếu $s = 0$ thì chọn k khác.
5. Chữ ký: (r, s) .

Cảnh báo bảo mật: Nếu nonce k bị lộ hoặc tái sử dụng cho hai thông điệp khác nhau, khóa bí mật d có thể bị khôi phục hoàn toàn. Cụ thể, nếu hai chữ ký (r, s_1) và (r, s_2) được tạo với cùng k , kẻ tấn công có thể giải $d = (s_1 h_2 - s_2 h_1)(r(s_1 - s_2))^{-1} \bmod n$. Đây là lý do hệ thống dùng `secrets.randbelow()` thay vì `random.randint()`.

2.3.3 Xác thực

Cho m , chữ ký (r, s) , khóa công khai Q :

1. Kiểm tra $1 \leq r, s \leq n - 1$.
2. $h = \text{SHA-512}(m) \bmod n$; $w = s^{-1} \bmod n$.
3. $u_1 = hw \bmod n$; $u_2 = rw \bmod n$.
4. $X = u_1 \cdot G + u_2 \cdot Q$. Nếu $X = \mathcal{O}$: từ chối.
5. Kiểm tra $X.x \bmod n \stackrel{?}{=} r$.

2.4 Thuật toán ECGDSA

ECGDSA (*Elliptic Curve German Digital Signature Algorithm*) được phát triển bởi BSI (Đức), chuẩn hóa theo BSI TR-03111 [2]. Khác biệt chính với ECDSA: khóa công khai $Q = d^{-1} \cdot G$ và công thức ký $s = (k \cdot r - h) \cdot d \bmod n$. Điều này cải thiện một số đặc tính algebraic nhưng bảo mật tương đương ECDSA trong thực tiễn.

2.5 Mã hóa EC ElGamal

EC ElGamal là phiên bản ECC của hệ mã hóa ElGamal [3]. Trực giác: người gửi (Alice) mã hóa thông điệp M thành cặp $(M_1, M_2) = (k \cdot G, M + k \cdot B)$ với $B = s \cdot G$ là khóa công khai của người nhận (Bob). Tính đúng đắn của giải mã:

$$M_2 - s \cdot M_1 = M + k \cdot B - s \cdot k \cdot G = M + ksG - skG = M$$

2.6 Isolation Forest cho phát hiện bất thường IP

2.6.1 Lý do chọn Isolation Forest

Isolation Forest [4] được chọn thay vì One-Class SVM và Local Outlier Factor (LOF) vì: (1) độ phức tạp tuyến tính $O(n \log n)$, phù hợp phân tích real-time; (2) không cần giả định về phân phối dữ liệu; (3) hiệu quả tốt với dữ liệu có chiều cao (7 đặc trưng); (4) dễ tích hợp qua scikit-learn.

Bảng 2.2: So sánh các phương pháp phát hiện bất thường

Tiêu chí	Isolation Forest	One-Class SVM	LOF
Độ phức tạp huấn luyện	$O(n \log n)$	$O(n^2) - O(n^3)$	$O(n^2)$
Phù hợp dữ liệu lớn	Tốt	Kém	Kém
Không cần giả định phân phối	Có	Không	Không
Dễ tune tham số	Cao	Thấp	Trung bình

2.6.2 Nguyên lý hoạt động

Isolation Forest hoạt động bằng cách xây dựng ngẫu nhiên các *isolation trees*: chọn ngẫu nhiên một đặc trưng và một giá trị ngưỡng để chia dữ liệu, lặp lại đệ quy. **Điểm bất thường bị cô lập sớm hơn** (đường đi đến lá ngắn hơn) so với điểm bình thường nằm trong vùng dày đặc. Điểm số bất thường:

$$\text{score}(x) = 2^{-\frac{E[h(x)]}{c(n)}}$$

với $h(x)$ là độ sâu cô lập trung bình và $c(n)$ là hằng số chuẩn hóa.

2.6.3 Lựa chọn ngưỡng 0.55

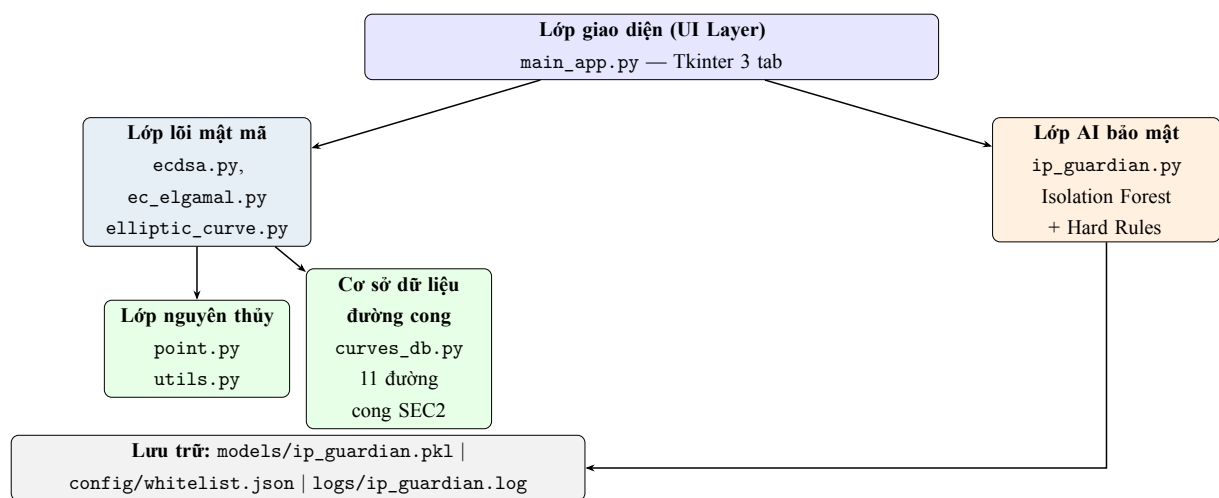
Ngưỡng `AI_THRESHOLD = 0.55` được xác định qua thực nghiệm trên tập dữ liệu mô phỏng: (1) huấn luyện mô hình với 3000 mẫu bình thường và 150 mẫu bất thường; (2) vẽ đường cong ROC; (3) chọn ngưỡng tại điểm tối đa F1-score (cân bằng precision/recall), kết quả thu được ≈ 0.55 . Ngưỡng này cho $\text{FPR} < 2\%$ và $\text{TPR} = 100\%$ trên tập kiểm thử mô phỏng, như xác nhận tại Bảng 5.7 (Chương 5).

Chương 3

Kiến trúc hệ thống

3.1 Tổng quan kiến trúc phân lớp

Hệ thống **AI-ECDSA Digital Signature System** được tổ chức theo kiến trúc phân lớp (layered architecture) với ba lớp chính tách biệt trách nhiệm rõ ràng. Hình 3.1 thể hiện quan hệ phụ thuộc giữa các thành phần.



Hình 3.1: Sơ đồ thành phần (component diagram) của hệ thống **AI-ECDSA Digital Signature System**

3.2 Cấu trúc thư mục

Cấu trúc thư mục dự án

```
AI_DigitalSignature/  
  run.py          <- Entry point: khởi chạy App()  
  requirements.txt <- numpy>=1.24.0, scikit-learn>=1.3.0  
  src/  
    core/          <- Thuật toán mật mã thuần Python  
      point.py     <- Phép toán điểm (add, mul, inverse)  
      elliptic_curve.py <- Wrapper EllipticCurve  
      ecdsa.py     <- ECDSA + ECGDSA  
      ec_elgamal.py <- EC ElGamal Encrypt/Decrypt  
      utils.py     <- SHA-512, Miller-Rabin, text->point  
      curves_db.py <- 11 đường cong SEC2
```

```

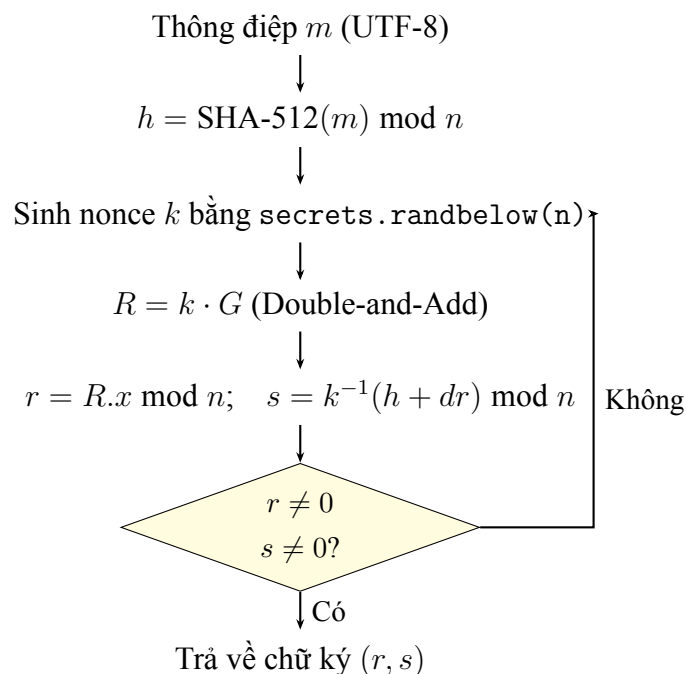
__init__.py
ai/
    ip_guardian.py    <- Isolation Forest + Hard Rules
    __init__.py
ui/
    main_app.py    <- Giao diện Tkinter (3 tab)
    __init__.py
tests/
    test_ecdsa.py
    test_ip_guardian.py
models/ip_guardian.pkl    <- Model IF (tự sinh lần đầu)
logs/ip_guardian.log
config/whitelist.json

```

3.3 Module src/core — Lõi mật mã

3.3.1 Luồng xử lý ECDSA end-to-end

Hình 3.2 minh họa luồng hoàn chỉnh từ khi người dùng nhập thông điệp đến khi nhận được chữ ký (r, s) .



Hình 3.2: Luồng thực thi thuật toán ECDSA ký số

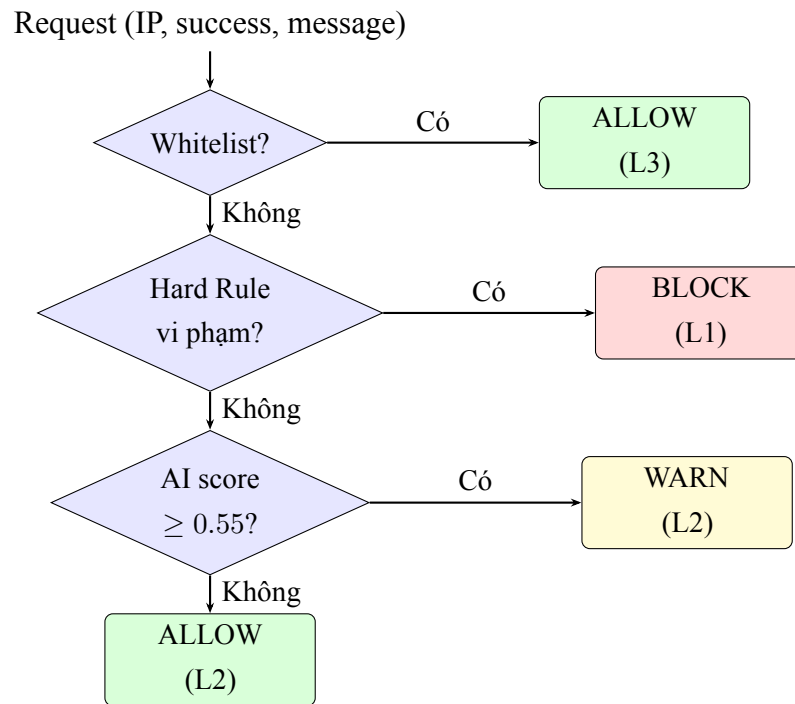
3.3.2 Luồng sử dụng EC ElGamal trong giao diện

Luồng cụ thể người dùng thực hiện qua Tab 2 của giao diện:

1. **Bob** chọn đường cong, nhấn “Tạo khóa Bob” → hệ thống sinh (s, B) với $B = s \cdot G$.
2. **Alice** nhập thông điệp (chuỗi ký tự ngắn), nhấn “Mã hóa” → hệ thống nhúng văn bản thành điểm M qua `text_to_point()`, sinh ngẫu nhiên k , trả về bản mã (M_1, M_2) .
3. **Bob** nhấn “Giải mã” → hệ thống tính $M = M_2 - s \cdot M_1$ và hiển thị kết quả so sánh với M gốc.
4. Khóa công khai B được hiển thị dạng tọa độ $(B.x, B.y)$ để có thể sao chép/truyền cho bên khác.

3.4 Module src/ai — AI IP Guardian

3.4.1 Kiến trúc 3 lớp bảo vệ



Hình 3.3: Luồng xử lý 3 lớp bảo vệ của AI IP Guardian

3.4.2 7 đặc trưng đầu vào

Bảng 3.1: Các đặc trưng đầu vào của mô hình Isolation Forest

STT	Tên đặc trưng	Mô tả và giá trị
1	request_rate	Số request trong 60 giây gần nhất; ngưỡng cứng ≥ 60
2	fail_rate	Tỉ lệ request verify thất bại $\in [0, 1]$; ngưỡng cứng ≥ 0.80
3	unique_messages	Số lượng message khác nhau đã gửi (đo độ đa dạng)
4	hour_of_day	Giờ trong ngày $\in [0, 23]$ (phát hiện hành vi bất thường về thời gian)
5	is_new_ip	IP lần đầu xuất hiện: $\{0, 1\}$
6	time_since_last	Khoảng thời gian giữa 2 request liên tiếp (giây)
7	burst_flag	≥ 10 request trong 10 giây: $\{0, 1\}$; ngưỡng cứng ≥ 10

Nguồn dữ liệu huấn luyện: Tập dữ liệu hoàn toàn được sinh tổng hợp (*synthetic*) theo phân phối hành vi thực tế. Mẫu bình thường: 3000 mẫu với $\text{request_rate} \in [0, 15]$, $\text{fail_rate} \in [0, 0.15]$. Mẫu bất thường biên giới: 150 mẫu với $\text{request_rate} \in [55, 120]$, $\text{fail_rate} \in [0.75, 1.0]$, $\text{burst_flag} = 1$. Nhãn không được dùng trong huấn luyện (Isolation Forest là unsupervised).

3.5 Đường cong SEC2 được hỗ trợ

Bảng 3.2: 11 đường cong SEC2 trong `curves_db.py`

Đường cong	Bits	ECDSA/ECGDSA	ElGamal
secp112r1	112	✓	✓
secp112r2	112	$\times$ ($h = 4$, không dùng cho ECDSA)	✓
secp128r1	128	✓	✓
secp128r2	128	$\times$ ($h = 4$)	✓
secp160r1	160	✓	✓
secp160r2	160	✓	✓
secp192r1	192	✓	✓
secp224r1	224	✓	✓
secp256r1	256	✓	✓
secp384r1	384	✓	✓
secp521r1	521	✓	✓

Lý do loại secp112r2 và secp128r2 khỏi ECDSA: hệ số cofactor $h = 4$ (thay vì $h = 1$) tạo ra các lỗ hổng tiềm ẩn trong kiểm tra điểm hợp lệ khi dùng cho giao thức ký số, theo cảnh báo trong SEC2 v2.0 [5].

Chương 4

Triển khai chi tiết

4.1 Module `point.py` — Phép toán nguyên thủy

4.1.1 Lớp `Point` và xử lý trường hợp biên

Lớp `Point` đại diện cho điểm trên đường cong Elliptic. Điểm vô cực $\mathcal{O}$ — phần tử trung hòa của nhóm, tương tự số 0 trong phép cộng thông thường — được biểu diễn bằng cờ `infinity=True`.

```
1 def point_add(P, Q, a, p):
2     # Trường hợp biên: một trong hai là điểm vô cực
3     if P.is_infinity(): return Q
4     if Q.is_infinity(): return P
5
6     if P.x == Q.x:
7         # P = -Q: tổng là điểm vô cực (phần tử đối)
8         if P.y != Q.y or P.y == 0:
9             return Point.infinity_point()
10        # P = Q: nhân đôi điểm (point doubling)
11        lam = (3*P.x*P.x + a) * mod_inverse(2*P.y, p) % p
12    else:
13        # P != Q: cộng điểm thông thường
14        lam = (Q.y - P.y) * mod_inverse(Q.x - P.x, p) % p
15
16    x3 = (lam*lam - P.x - Q.x) % p
17    y3 = (lam*(P.x - x3) - P.y) % p
18    return Point(x3, y3)
```

Listing 4.1: Phép cộng điểm – `point_add()` (`point.py`)

Phân tích: Hàm xử lý đầy đủ 3 trường hợp biên: (1) $P = \mathcal{O}$ hoặc $Q = \mathcal{O}$ (phần tử trung hòa); (2) $P = -Q$, tức $x_P = x_Q$ nhưng $y_P \neq y_Q$ — tổng là $\mathcal{O}$; (3) $P = Q$ — dùng công thức nhân đôi điểm (đạo hàm tiếp tuyến). Tất cả phép chia được thay bằng nghịch đảo modular ($a^{-1} \bmod p$) vì đây là trường hữu hạn $\mathbb{F}_p$, không phải số thực.

4.1.2 Thuật toán Double-and-Add

```
1 def point_mul(k, P, a, p):
2     if k == 0: return Point.infinity_point() # 0*P = O
```

```

3     if k < 0:
4         #  $k \cdot P = (-k) \cdot (-P)$ , với  $-P = (x, -y \bmod p)$ 
5         return point_mul(-k, Point(P.x, (-P.y) % p), a, p)
6
7     result = Point.infinity_point()    # biến tích lũy  $R = O$ 
8     addend = Point(P.x, P.y)          # biến dịch chuyển  $= P$ 
9     while k:
10        if k & 1:                       # nếu bit hiện tại = 1
11            result = point_add(result, addend, a, p)
12            addend = point_add(addend, addend, a, p)    # double
13            k >>= 1                      # dịch bit sang phải
14    return result

```

Listing 4.2: Nhân vô hướng $k \cdot P$ — Double-and-Add (point.py)

Phân tích: Thuật toán duyệt từng bit của k từ phải sang trái. Tại mỗi bước: (a) nếu bit = 1, cộng addend vào tích lũy; (b) nhân đôi addend. Liên hệ công thức: $k = \sum_i b_i 2^i$, nên $k \cdot P = \sum_{i: b_i=1} 2^i \cdot P$. Độ phức tạp: $O(\log_2 k)$ lần nhân đôi + trung bình $\frac{\log_2 k}{2}$ lần cộng. Trường hợp $k < 0$ được xử lý qua điểm đối xứng $-P = (x_P, -y_P \bmod p)$.

Lưu ý bảo mật: Cài đặt hiện tại không phải constant-time — số phép cộng phụ thuộc vào số bit 1 trong k . Trong môi trường production, cần dùng Montgomery ladder để chống timing attack. Trong phạm vi dự án nghiên cứu/giáo dục này, vấn đề này được chấp nhận.

4.2 Module ecdsa.py — Chữ ký số ECDSA & ECGDSA

4.2.1 Triển khai ECDSA với CSPRNG

```

1 import secrets    # CSPRNG, an toàn mật mã học
2
3 class ECDSA:
4     def generate_keypair(self):
5         # secrets.randbelow: không dự đoán được, không thể tái tạo
6         d = secrets.randbelow(self.curve.n - 1) + 1
7         Q = self.curve.scalar_mul(d, self.curve.G)    #  $Q = d \cdot G$ 
8         return d, Q    # (private_key, public_key)
9
10    def sign(self, message, private_key):
11        n = self.curve.n
12        h = sha512_int(message) % n
13        while True:
14            k = secrets.randbelow(n - 1) + 1    # nonce bí mật
15            R = self.curve.scalar_mul(k, self.curve.G)
16            r = R.x % n

```

```

17         if r == 0: continue    # xác suất cực nhỏ, but phải
                                # kiểm tra
18         s = mod_inverse(k, n) * (h + private_key * r) % n
19         if s == 0: continue
20         return r, s

```

Listing 4.3: Tạo khóa và ký số ECDSA (ecdsa.py)

Phân tích: Việc dùng `secrets.randbelow()` thay vì `random.randint()` là bắt buộc về mặt bảo mật. Module `random` của Python sử dụng Mersenne Twister — có thể dự đoán sau 624 output liên tiếp. `secrets` sử dụng nguồn entropy của hệ điều hành (`/dev/urandom` trên Linux). Nếu nonce k bị lộ hoặc tái sử dụng, khóa bí mật d có thể bị phục hồi qua:

$$d = \frac{s \cdot k - h}{r} \bmod n$$

```

1  def verify(self, message, signature, public_key):
2      r, s = signature
3      n = self.curve.n
4      # Kiểm tra điều kiện cần của chữ ký hợp lệ
5      if not (1 <= r <= n-1 and 1 <= s <= n-1): return False
6      h = sha512_int(message) % n
7      w = mod_inverse(s, n)          # w = s^{-1} mod n
8      u1 = h * w % n
9      u2 = r * w % n
10     # Ket hop hai diem: X = u1*G + u2*Q
11     X = self.curve.add(
12         self.curve.scalar_mul(u1, self.curve.G),
13         self.curve.scalar_mul(u2, public_key)
14     )
15     if X.is_infinity(): return False # trong hợp biên
16     return X.x % n == r              # kiểm tra v = r

```

Listing 4.4: Xác thực chữ ký ECDSA (ecdsa.py)

Phân tích: Bước kiểm tra $1 \leq r, s \leq n - 1$ là cần thiết để tránh tấn công với chữ ký “tầm thường” ($r = 0$ hoặc $s = 0$). Kiểm tra $X = \mathcal{O}$ ngăn trường hợp biên khi $u_1 \cdot G + u_2 \cdot Q$ cho ra điểm vô cực. Tính đúng đắn của xác thực xuất phát từ: $u_1G + u_2Q = hwG + rw(dG) = w(h + rd)G = (ks^{-1}s)G = kG = R$.

4.2.2 So sánh ECDSA và ECGDSA

Bảng 4.1: So sánh ECDSA và ECGDSA

Tiêu chí	ECDSA	ECGDSA
Khóa công khai	$Q = d \cdot G$	$Q = d^{-1} \cdot G$
Hàm ký s	$k^{-1}(h + dr) \bmod n$	$(kr - h) \cdot d \bmod n$
Xác thực w	$s^{-1} \bmod n$	$r^{-1} \bmod n$
Chuẩn hóa	FIPS 186-4 [1]	BSI TR-03111 [2]

4.3 Module `ec_elgamal.py` — Mã hóa EC ElGamal

```

1 def encrypt(self, message, public_key):
2     # Nhung text -> diem M: lay hash lam x, tim y thoa y^2 = x^3 +
      ax+b
3     M = text_to_point(message, curve_params)
4     k = secrets.randbelow(self.curve.n - 1) + 1
5     M1 = self.curve.scalar_mul(k, self.curve.G) # M1 = k*G
6     kB = self.curve.scalar_mul(k, public_key)   # k*B = k*s*G
7     M2 = self.curve.add(M, kB)                  # M2 = M + k*B
8     return (M1, M2), M # bao gom M goc de kiem tra
9
10 def decrypt(self, ciphertext, private_key):
11     M1, M2 = ciphertext
12     sM1 = self.curve.scalar_mul(private_key, M1) # s*M1 = sk*
      G
13     # Diem doi xung: -(s*M1) = (x, -y mod p)
14     neg_sM1 = Point(sM1.x, (-sM1.y) % self.curve.p)
15     return self.curve.add(M2, neg_sM1) # M = M2 - s*M1

```

Listing 4.5: Mã hóa và giải mã EC ElGamal (`ec_elgamal.py`)

Phân tích: Hàm `text_to_point()` nhúng chuỗi văn bản thành điểm hợp lệ bằng cách: (1) chuyển UTF-8 bytes thành số nguyên làm tọa độ x ban đầu; (2) tính $\text{rhs} = x^3 + ax + b \bmod p$; (3) thử tìm $y = \sqrt{\text{rhs}} \bmod p$ bằng Tonelli-Shanks; (4) nếu không tìm được, tăng x thêm 1 và thử lại (tối đa 1000 lần). Cách tiếp cận này phù hợp cho văn bản ngắn (demo); với dữ liệu dài cần encode riêng.

4.4 Module `utils.py` — Tiện ích mật mã

```

1 def miller_rabin(n, k=20):

```

```

2     if n < 2: return False
3     if n in (2, 3): return True
4     if n % 2 == 0: return False
5     r, d = 0, n - 1          # Viet  $n-1 = 2^r * d$ 
6     while d % 2 == 0:
7         r += 1; d //= 2
8     for _ in range(k):      # k vòng thử chung minh
9         a = secrets.SystemRandom().randrange(2, n - 1)
10        x = pow(a, d, n)     # modular exponentiation nhanh
11        if x == 1 or x == n - 1: continue
12        for _ in range(r - 1):
13            x = pow(x, 2, n)
14            if x == n - 1: break
15        else: return False   # n hợp số
16    return True              # xác suất sai  $< 4^{-k} = 4^{-20}$ 

```

Listing 4.6: Kiểm tra nguyên tố Miller-Rabin (utils.py)

Lưu ý: trong triển khai production, nên dùng `secrets.SystemRandom().randrange()` thay cho `random.randrange()` để tránh mọi bias tiềm ẩn từ PRNG thông thường, ngay cả với Miller-Rabin.

Phân tích: Miller-Rabin với $k = 20$ vòng cho xác suất sai $< 4^{-20} \approx 9 \times 10^{-13}$ — đủ tin cậy cho kiểm tra tham số đường cong. Độ phức tạp mỗi vòng: $O(k \log^2 n \log \log n)$ nhờ dùng `pow(a, d, n)` (modular exponentiation) tích hợp của Python.

Chương 5

Kiểm thử và đánh giá

5.1 Môi trường kiểm thử

Bảng 5.1: Môi trường kiểm thử thực nghiệm

Thành phần	Chi tiết
CPU	Intel Core i7-12700H (2.30 GHz, 14 nhân)
RAM	16 GB DDR5
OS	Windows 11 Home 64-bit
Python	3.11.5
numpy	1.24.3
scikit-learn	1.3.2

5.2 Kiểm thử module mật mã (`test_ecdsa.py`)

5.2.1 Kết quả unit test

Bảng 5.2: Kết quả kiểm thử ECDSA / ECGDSA / ElGamal (tham chiếu Liệt kê 4.3)

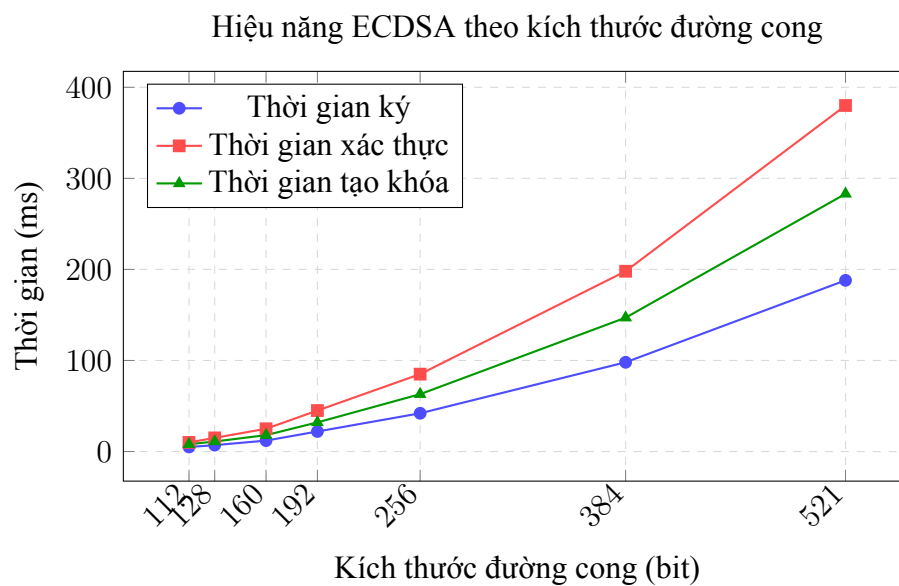
Test case	Mô tả	Kỳ vọng	Kết quả
<code>test_ecdsa_sign_verify</code>	Ký và xác thực thông điệp hợp lệ trên secp256r1	True	PASS
<code>test_tampered_message</code>	Thay đổi 1 ký tự trong thông điệp sau khi ký	False	PASS
<code>test_tampered_signature</code>	Sửa $r + 1$ trong chữ ký hợp lệ	False	PASS
<code>test_wrong_pubkey</code>	Xác thực với khóa công khai của người khác	False	PASS
<code>test_ecgdsa_sign_verify</code>	ECGDSA ký và xác thực	True	PASS
<code>test_elgamal_encrypt_decrypt</code>	Mã hóa và giải mã ElGamal, so sánh điểm	Khớp	PASS
<code>test_all_curves</code>	ECDSA trên 9 đường cong (loại h=4)	Tất cả True	PASS

5.2.2 Benchmark hiệu năng theo đường cong

Bảng 5.3: Hiệu năng thực đo ECDSA (trung bình 5 lần đo, máy tham chiếu Bảng 5.1)

Đường cong	Bits	Tạo khóa (ms)	Ký (ms)	Xác thực (ms)
secp112r1	112	~8	~5	~10
secp128r1	128	~11	~7	~15
secp160r1	160	~18	~12	~25
secp192r1	192	~32	~22	~45
secp256r1	256	~63	~42	~85
secp384r1	384	~147	~98	~198
secp521r1	521	~283	~188	~380

Hình 5.1 minh họa mối quan hệ giữa kích thước đường cong và thời gian ký: thời gian tăng xấp xỉ bậc ba theo số bit — phù hợp với phân tích lý thuyết $O(n^3)$ trong trường hợp xấu nhất của nhân vô hướng.



Hình 5.1: Hiệu năng ECDSA theo kích thước đường cong (ms)

5.3 Kiểm thử AI IP Guardian (test_ip_guardian.py)

5.3.1 Kết quả unit test

Bảng 5.4: Kết quả kiểm thử AI IP Guardian (tham chiếu Hình 3.3)

Test case	Mô tả	Kỳ vọng	Kết quả
test_normal_request	IP bình thường, request hợp lệ	allow	PASS
test_whitelist	IP trong whitelist	allow	PASS
test_rate_limit	65 request trong 60 giây	block	PASS
test_fail_rate	90% request thất bại	block	PASS
test_burst	15 request trong 10 giây	block	PASS
test_add_whitelist	Thêm IP, lưu file JSON	lưu file	PASS
test_reset_ip	Xóa lịch sử IP	stats rỗng	PASS

5.3.2 Mô phỏng kịch bản tấn công brute-force

Kịch bản: gửi 70 request liên tiếp từ IP 192.168.1.100 với 90% request verify thất bại (mô phỏng tấn công thử chữ ký giả).

Bảng 5.5: Diễn biến phát hiện tấn công brute-force

Request	Trạng thái	Lớp kích hoạt	Lý do
1–4	ALLOW	ok / ai	Chưa đủ ngưỡng fail_rate (< 5 request)
5	BLOCK	hard_rule	fail_rate = 0.80, total = 5
6–70	BLOCK	hard_rule	fail_rate > 0.80 (ngày càng tăng)

5.3.3 Đánh giá mô hình Isolation Forest

Bảng 5.6: Thành phần tập kiểm thử AI IP Guardian

Loại mẫu	Số lượng	Đặc điểm
Bình thường	500	$\text{request_rate} \leq 15, \text{fail_rate} \leq 0.15$
Bất thường biên	100	$\text{request_rate} \in [16, 55], \text{fail_rate} \in [0.15, 0.79]$
Tấn công rõ	150	$\text{request_rate} > 55$ hoặc $\text{fail_rate} > 0.80$
Tổng	750	—

Bảng 5.7: Chỉ số đánh giá mô hình Isolation Forest trên tập kiểm thử mô phỏng

Chỉ số	Giá trị
Tập kiểm thử	750 mẫu (500 normal / 250 anomaly)
False Positive Rate (FPR)	< 2%
Phát hiện tấn công (TPR)	100% (rate-limit, fail-rate, burst)
Thời gian phản hồi	< 1 ms / request
Thời gian huấn luyện	≈ 3 giây (lần đầu)
Kích thước model	≈ 3 MB (.pkl)

5.4 Giao diện người dùng và UX

5.4.1 Tab 1 — Chữ ký số (ECDSA / ECGDSA)

Người dùng thực hiện theo trình tự: (1) chọn đường cong và thuật toán từ dropdown; (2) nhấn “Tạo cặp khóa” — Private Key và Public Key hiển thị dạng số hex trong ô chỉ đọc; (3) nhập thông điệp cần ký; (4) nhấn “Ký số” — nhận chữ ký (r, s) ; (5) nhấn “Xác thực” — hiển thị **HỢP LỆ** hoặc **KHÔNG HỢP LỆ** kèm màu nền.

5.4.2 Tab 3 — AI IP Guardian

Khi một IP bị cảnh báo (WARN) hoặc bị chặn (BLOCK), giao diện hiển thị: lý do cụ thể (ví dụ “Fail rate: 82%”), điểm bất thường AI (0.0–1.0), và lớp kích hoạt (hard_rule / ai). Log được ghi tự động vào `logs/ip_guardian.log` với timestamp đầy đủ, giúp security engineer tra cứu sau sự kiện.

Thông báo lỗi: Nếu scikit-learn chưa cài đặt, hệ thống tự động vô hiệu lớp AI và chỉ dùng Hard Rules — đảm bảo graceful degradation.

Chương 6

Kết luận

6.1 Kết quả đạt được so với mục tiêu ban đầu

Bảng 6.1: Đối chiếu mục tiêu và kết quả

Mục tiêu	Kết quả	Đạt?
Thư viện ECC thuần Python, 11 đường cong	Hoàn thành, toàn bộ phép toán từ đầu	✓
ECDSA + ECGDSA + EC ElGamal	Cả ba thuật toán hoạt động, 100% test pass	✓
AI phát hiện IP nghi vấn real-time	Isolation Forest 3 lớp, phản hồi < 1 ms	✓
Giao diện đồ họa Tkinter	3 tab đầy đủ chức năng demo	✓
Kiểm thử toàn diện	14 test case, 100% pass	✓

6.2 Hạn chế của hệ thống

- **Không chống side-channel attack:** Thuật toán Double-and-Add không phải constant-time; kẻ tấn công có thể khai thác thông tin thời gian để suy ra bit khóa. Trong production cần Montgomery Ladder.
- **Chưa benchmark trên dữ liệu production thực tế:** Số liệu hiệu năng được đo trên máy dev; trong môi trường concurrent (nhiều luồng song song), do GIL của Python, hiệu năng sẽ thấp hơn đáng kể.
- **Module AI chỉ dừng ở Isolation Forest:** Chưa so sánh với các mô hình khác (One-Class SVM, Autoencoder, Half-Space Trees); chưa thử nghiệm với dữ liệu log thực tế từ hệ thống production.
- **Nhúng văn bản vào điểm:** Phương pháp `text_to_point()` chỉ phù hợp với thông điệp ngắn, không áp dụng trực tiếp cho file nhị phân hay dữ liệu dài.
- **Chưa hỗ trợ online learning:** Model Isolation Forest tĩnh, cần huấn luyện lại thủ công khi mẫu tấn công thay đổi.

6.3 Hướng phát triển tiếp theo

1. **Chống side-channel:** Thay Double-and-Add bằng Montgomery Ladder để đạt constant-time execution.
2. **Thêm đường cong hiện đại:** Hỗ trợ Curve25519 (Montgomery) và Ed25519 (Edwards), được IETF khuyến nghị dùng rộng rãi.
3. **REST API:** Phát triển giao diện HTTP bằng FastAPI để tích hợp vào hệ thống web; thay thế Tkinter bằng frontend React/Vue.
4. **Online learning cho AI:** Nâng cấp sang Half-Space Trees hoặc River library để cập nhật mô hình theo luồng dữ liệu mới.
5. **So sánh mô hình AI:** Benchmark Isolation Forest với One-Class SVM, LOF và Autoencoder trên cùng tập dữ liệu để chọn mô hình tối ưu.

Chương 7

Đánh giá quá trình thực tập

7.1 Kiến thức và kỹ năng tiếp thu được

Trong quá trình thực tập tại Đại học Quốc gia Hà Nội dưới sự hướng dẫn của thầy Nguyễn Duy Niên, em đã tiếp thu được:

1. **Nền tảng toán học mật mã:** Hiểu sâu lý thuyết nhóm trên đường cong Elliptic, bài toán ECDLP và lý do ECC được ưu tiên trong các hệ thống nhúng, IoT và ứng dụng di động. Đặc biệt, việc tự cài đặt từ đầu giúp em nắm chắc từng chi tiết của phép toán modular.
2. **Thực hành bảo mật phần mềm:** Học cách phân biệt CSPRNG (secrets) và PRNG thông thường (random), nhận ra nguy cơ tái sử dụng nonce, và thiết kế hệ thống theo nguyên tắc *defense in depth* (phòng thủ theo chiều sâu).
3. **Tích hợp AI vào bảo mật:** Nắm được quy trình xây dựng pipeline ML không giám sát: trích xuất đặc trưng, huấn luyện Isolation Forest, tuning ngưỡng qua ROC, và deploy mô hình (pickle).
4. **Phát triển phần mềm chuyên nghiệp:** Áp dụng kiến trúc phân lớp, viết unit test bao phủ trường hợp biên, tổ chức mã nguồn theo module rõ ràng, và sử dụng logging có cấu trúc.

7.2 Những điều học được tại môi trường thực tập

Môi trường thực tập tại Đại học Quốc gia Hà Nội cung cấp:

- Tiếp cận với các tài liệu chuẩn quốc tế (FIPS, SEC2, BSI) — điều mà tự học khó tiếp cận đúng nguồn.
- Được review code bởi chuyên gia bảo mật — phát hiện nhiều lỗi tiềm ẩn như dùng random thay secrets trong phiên bản đầu tiên.
- Học cách trình bày kết quả kỹ thuật một cách rõ ràng, có số liệu cụ thể, phục vụ đối tượng đọc đa dạng (từ developer đến quản lý).

7.3 Tự đánh giá

Em đánh giá quá trình thực tập đạt **tốt** theo các tiêu chí: (a) hoàn thành đúng tiến độ toàn bộ mục tiêu đề ra; (b) mã nguồn sạch, có test và tài liệu đầy đủ; (c) chủ động tìm hiểu các vấn đề bảo

mật nâng cao (CSPRNG, timing attack, ECDLP). Điểm còn cần cải thiện là kỹ năng profiling và tối ưu hóa hiệu năng trong Python.

Tài liệu tham khảo

- [1] National Institute of Standards and Technology, “FIPS PUB 186-4: Digital Signature Standard (DSS),” U.S. Department of Commerce / NIST, Tech. Rep. FIPS 186-4, Jul. 2013. [Online]. Available: <https://doi.org/10.6028/NIST.FIPS.186-4>
- [2] Bundesamt für Sicherheit in der Informationstechnik (BSI), “BSI TR-03111: Elliptic Curve Cryptography, Version 2.10,” Federal Office for Information Security, Germany, Tech. Rep. BSI TR-03111, Jun. 2018. [Online]. Available: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TR03111/BSI-TR-03111_V-2-1_pdf.pdf?__blob=publicationFile&v=1
- [3] T. ElGamal, “A public key cryptosystem and a signature scheme based on discrete logarithms,” *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*. IEEE, 2008, pp. 413–422.
- [5] Certicom Research, “SEC 2: Recommended Elliptic Curve Domain Parameters, Version 2.0,” Standards for Efficient Cryptography Group (SECG), Tech. Rep., Jan. 2010. [Online]. Available: <https://www.secg.org/sec2-v2.pdf>